

Contents

AI Coding 最佳实践开发 SOP	2
修订版 v11	2
基于 OCN 两阶段模型 + 三主干（逻辑 • 就绪 • 任务）+ Claude Code 自动接线 + 可选自动模式	2
一句话总结	2
目录	2
第一章 核心原则	3
1.1 先定义，再实现	3
1.2 前半段强门禁，后半段强证据	3
1.3 文档不是目的，工程闭环才是目的	3
1.4 真实实现证据优先于事后叙述	3
1.5 AI 是执行放大器，不是责任替代者	3
1.6 先接好逻辑主干，再写代码	4
1.7 没有角色签收，就没有完成	4
1.8 收据不是现实，勾销只认验收命令	4
1.9 四句话带走	4
第二章 完整操作指引（分阶段命令手册）	5
2.1 安装 / 升级 / 卸载	5
2.2 项目启动（一次性）	5
2.3 第一阶段：Planning Gate（00 → 11）	6
2.4 就绪主干：何时用哪条命令	6
2.5 第二阶段：Execution Navigator（11 之后）	7
2.6 回拨与重开：游标的受控移动	7
2.7 自动模式：把推进触发委托给 AI（可选，默认关闭）	8
2.8 全流程速查（一页版）	9
第三章 两阶段总览（Planning Gate • Execution Navigator）	10
3.1 总体结构	10
3.2 两阶段之间的衔接点	11
3.3 角色变化	11
3.4 第一阶段：Planning Gate	11
3.5 00 项目简报	11
3.6 01 范围	11
3.7 02 PRD	12
3.8 03 验收标准	12
3.9 04 技术架构	12
3.10 05 到 09 设计层文档	12
3.11 逻辑主干（Logic Backbone）	13
3.12 11 Build Plan	13
3.13 就绪主干（Readiness Backbone）	14
3.14 为什么第二阶段不能继续做成线性文档推进	15
3.15 第二阶段的事实来源	16
3.16 第二阶段不是取消文档，而是改变文档角色	16
第四章 完整文档体系	16
4.1 第一阶段核心文档	16
4.2 第二阶段可承接文档	16
4.3 本版推荐理解	17
第五章 阶段门禁与推进规则	17
5.1 第一阶段推进规则	17
5.2 第二阶段推进规则	17
5.3 判断逻辑	17

第六章 AI 与人的分工 18

6.1 AI 适合做什么 18

6.2 人必须保留什么 18

6.3 不应让 AI 做什么 18

第七章 常见错误与纠偏建议 18

7.1 错误一：还没定义清楚就急着进入实现 18

7.2 错误二：进入开发后继续把 OCN 当作文档推进器 19

7.3 错误三：重复手工记录 GitHub 已经存在的证据 19

7.4 错误四：把 AI 当作最终裁判 19

7.5 错误五：把 evidence-candidate 当成 evidence-found 19

7.6 错误六：在 partial 状态下强行进入 merge 19

7.7 错误七：文档结构齐全，但逻辑未接线 19

7.8 错误八：角色盲区的假完成 19

7.9 错误九：收据型假完成与全链空转 19

第八章 修订说明 20

阶段一 20

阶段二 20

v3 升级：补上”逻辑主干” 20

v4 修订 21

本次 v5 修订：补上”就绪主干” 21

本次 v6 修订：把接线产品化——纪律靠机械强制，不靠记忆 21

本次 v7 修订：补上”任务主干”——堵死第四类假完成 21

本次 v8 修订：补上”回拨与重开”——受控逃生通道 22

本次 v9 修订：里程碑循环——回拨的标准用法定型 22

本次 v10 修订：术语校正——“纠错”改”回拨与重开” 22

本次 v11 修订：自动模式——可选的触发权委托 22

AI Coding 最佳实践开发 SOP

修订版 v11

基于 OCN 两阶段模型 + 三主干（逻辑 · 就绪 · 任务）+ Claude Code 自动接线 + 可选自动模式

修订日期：2026-06-13

一句话总结

AI Coding 最佳实践，不再是单纯“先写文档，再写代码”，也不是“让 Agent 一路自动生成”，而是分成两个互相衔接的阶段：

前半段用强门禁把问题想清楚，后半段用证据导航把实现做扎实。

前半段防失控，后半段防失真；三根主干，把”假完成”挡在每一道门外。

目录

- 第一章 核心原则
- 第二章 完整操作指引（分阶段命令手册）
- 第三章 两阶段总览（Planning Gate · Execution Navigator）

- 第四章 完整文档体系
 - 第五章 阶段门禁与推进规则
 - 第六章 AI 与人的分工
 - 第七章 常见错误与纠偏建议
 - 第八章 修订说明
-

第一章 核心原则

1.1 先定义，再实现

任何 AI Coding 项目，真正昂贵的不是写出代码，而是：

- 问题没定义清楚
- 边界没锁定
- 验收没说清楚
- 架构没选定
- 数据与接口关系没理清
- 结果只能事后补救

因此，开发前必须先把手高不确定性问题转成结构化决策。

1.2 前半段强门禁，后半段强证据

前半段的目标不是“快写代码”，而是“快把输入压实”。

后半段的目标不是“继续堆文档”，而是“让执行证据可见、可审、可导航”。

1.3 文档不是目的，工程闭环才是目的

文档的价值在于：

- 让问题可判断
- 让门禁可执行
- 让 AI 有清晰输入
- 让后续证据有对照基线

如果文档不能服务于后续实现与验证，它就会退化为形式主义。

1.4 真实实现证据优先于事后叙述

进入开发阶段后，优先级最高的事实是：

- 改了哪些文件
- 哪些测试通过或失败
- 哪些 PR 已合并
- 哪些 review 仍未处理
- 哪些验收条目仍缺证据
- 当前是否具备进入 review 或 merge 的条件

所以，后半段要尽量减少重复手工记录，优先利用已经存在的工程证据链。

1.5 AI 是执行放大器，不是责任替代者

AI 可以帮助：

- 生成结构化文档

- 拆解任务
- 归纳证据
- 解释状态
- 生成下一轮提示词
- 草拟 verdict

但 AI 不应替代：

- 关键决策
- 验收裁定
- 风险承担
- 发布授权
- 产品取舍

1.6 先接好逻辑主干，再写代码

定义清楚”系统有哪些东西、长什么样”还不够。开工前还要定义清楚”这些东西按什么顺序、以什么角色连起来”：

- 先算什么、后算什么（执行顺序）
- 哪个公式服务哪个判断（公式归属）
- 哪个分数进入下一层、哪个分数只做解释（分数分层）
- 哪个信号能推动功能触发、哪个信号只是提示（信号作用级别）

这四件事就是系统的**逻辑主干**。它必须在设计层被显式画出来、并且可机器校验，否则实现一定漂移。详见 3.11。

1.7 没有角色签收，就没有完成

章节齐了、逻辑接线了，还不等于”准备好了”。一个项目最终要被一批具体的角色验收：开发要能跑起来、测试要有可证伪的验收、运维要有人对可运维性负责、安全要有最低基线、业务要有真实使用者。

- “缺了哪些维度”无法穷举，但”哪些角色必须签收”有边界、可编目
- 沉默不是通过：一项检查没有证据（UNKNOWN），就和失败（FAIL）一样阻断
- 确实不适用的检查，必须**显式豁免并留下理由**，而不是装作不存在

这就是**就绪主干**：把”内容完整吗”这个不可验证的命题，换成”每个必需角色都 PASS 或被显式豁免了吗”这个可验证的命题。详见 3.13。

1.8 收据不是现实，勾销只认验收命令

文档收据写得再诚实，也不等于实现发生了。实施必须被**显式排入**（任务规格）、被**机器验收**（每个任务自带的确定性验收命令）：

- 拆分发生在进 BUILD 之前：build plan 里每个任务是一份迷你 spec，拆得好不好先过门禁
- 验收命令在门禁通过时**冻结**——实施期间偷改裁判会被抓
- 勾销没有人工通道：命令 exit 0 才算 done
- **任务台账不清，不准出 BUILD**——“先走完流程再补码”在结构上不可能

这就是**任务主干**。详见 3.12。

1.9 四句话带走

以上八条原则，可以压成四句话随身带走：

- **先把问题想清楚，再用证据把实现做扎实。**——简版：1.1（先定义，再实现）与 1.4（真实证据优先于事后叙述）的合写。
- **Planning Gate 锁输入。Execution Navigator 读证据。**——工程版：1.2（前半段强门禁，后半段强证据）落到两阶段产品形态上的命名。

- **前半段防失控，后半段防失真。**——产品版：失控靠门禁挡、失真靠证据挡，对应 1.2 与 1.3（文档不是目的，工程闭环才是目的）。
- **不是让 Agent 一路写下去，而是让系统在关键节点知道现在发生了什么、下一步该做什么。**——AI Coding 版：1.5（AI 是执行放大器，不是责任替代者）加上 1.6–1.8 三根主干的机械防线，合成的交互节奏。

第二章 完整操作指引（分阶段命令手册）

本章是全书唯一的命令主场：每个阶段用什么命令、具体什么时候用，包括安装、升级与卸载。其余各章讲为什么，本章只讲怎么做、何时做。

2.1 安装 / 升级 / 卸载

本 SOP 由 OCN（O' CodingNavigator）落地。开工前先装好 ocn 与 ocn-mcp 两个命令。**前置依赖**：Node.js $\geq$ 20。

安装（npm 全局）：

```
npm install -g o-coding-navigation    # latest 通道，当前即 SOP 0.5.0
ocn --version                         # 验证：0.6.0-beta.0
ocn-mcp                               # 验证 MCP server 可启动；Ctrl+C 退出
```

如需固定在 beta 预发布通道：npm install -g o-coding-navigation@beta。贡献者可走源码路径：git clone https://github.com/UncleTIM-GZ/O-CodingNavigation.git && cd O-CodingNavigation && npm install && npm run build && npm link。

升级分两层，缺一不可：

① 升级工具本体（npm 包）

```
npm install -g o-coding-navigation@latest
```

② 升级已有项目的 SOP 锁定版本（在项目目录内执行）

```
ocn sop upgrade --plan                # 干跑：列出将要改动的快照文件
ocn sop upgrade                       # 执行：迁移到当前内置默认（0.5.0）
```

只做 ① 不做 ②，旧项目仍按初始化时锁定的旧 SOP 运行（这是设计行为，不是 bug）；进度游标、已写文档、config.yaml 自定义命令在 ② 中全部保留。仅支持向前升级。

卸载：

```
npm uninstall -g o-coding-navigation  # 卸载工具
rm -rf .ocoding                      # （可选）删除某项目的 OCN 状态；docs/ 下文档不受影响
```

完整安装步骤、ocn init 后的文件树与常见排障，见 docs/quickstart.md。

2.2 项目启动（一次性）

```
cd <你的项目>
ocn init --tier minimal                # 创建 .ocoding/ 与 docs/，锁定 SOP 0.5.0
ocn status                             # 确认当前位置：state_discovery / step_project_brief
```

init 之后**立刻**做一件事：在 .ocoding/config.yaml 的 commands: 段登记本项目的构建与测试命令——就绪检查靠它们取证，不登记则相关检查永远是 UNKNOWN（UNKNOWN 会阻断）：

```
commands:
  build: npm run build
  test: npm run test
  test_list: npx vitest list
```

然后跑一次就绪基线，知道自己离”可开工”还差哪些角色的证据：

ocn readiness list

最后一步（v6 新增）：**接线 Claude Code**——一条命令生成全部集成文件并入库共享：

```
ocn agent setup # 幂等；--force 仅用于修复损坏的 settings.json
git add .claude CLAUDE.md && git commit # 入库 → 全队克隆即生效，队友零配置
```

它生成四件套：.claude/settings.json 两条钩子（结束回合 → 自动跑 ocn check 门禁；每次编辑 → 自动跑 commands.lint/typecheck 反馈；带 command -v ocn 守卫，没装 ocn 的队友不受影响）、.claude/ocn.md 治理契约（随每个会话自动加载）、/ocn-next 斜杠命令。接线后，2.3/2.5 循环中”看简报、贴提示词、盯自查”的部分全部自动化，人只剩 /ocn-next 和 ocn advance 两个动作。

2.3 第一阶段：Planning Gate（00 → 11）

每一份文档都走同一个四步循环（已接线 Claude Code 时：输入 /ocn-next 即自动完成 ①② 并开始执行，④ 由 Stop 钩子在 agent 结束回合时强制运行）：

```
ocn brief # ① 看当前步骤要写什么、必需章节有哪些
ocn doc create <type> # ② 从模板创建文档
# （人 + AI 写内容） # ③ 填实内容
ocn check # ④ 三道门：章节 → 逻辑主干 → 就绪
ocn advance # 通过则推进到下一步（advance 自动先跑 gate）
```

<type> 按顺序依次是：

序	文档	ocn doc create <type>
00	项目简报	project-brief
01	范围	scope
02	PRD	prd
03	验收标准	acceptance-criteria
04	技术架构	technical-architecture
05	信息架构	information-architecture
06	数据模型	data-model
07	逻辑主干	logic-backbone
08	接口契约	api-contract
09	测试策略	test-strategy
10	MVP 计划	mvp-plan
11	Build Plan	build-plan

被 ocn check 阻断时看退出码：

- **exit 2**（产物问题）：缺必需章节、07 逻辑主干命中六类缺陷（缺角色/重复 id/悬空引用/环/孤儿/未绑定触发），或 **11 build plan 的任务规格命中六类缺陷**（重复/非法 id、缺字段、traces 悬空、touches 悬空、depends 悬空/成环、零任务）——回去补文档本身
- **exit 1**（门禁问题）：就绪检查未过——执行 ocn readiness list 看是哪些角色检查 FAIL/UNKNOWN，按 fix_hint 补证据，或确实不适用时显式豁免（见 2.4）

11 build plan 通过的瞬间，任务台账 .ocoding/task-ledger.json 生成、验收命令冻结——第一阶段结束，进入第二阶段的**任务循环**。

2.4 就绪主干：何时用哪条命令

就绪检查不需要”记得去跑”——它已内嵌在每次 check / gate / advance 里。需要主动执行的只有四个时机：

时机	命令
init 之后，建立基线	ocn readiness list
任何 check / advance 以 exit 1 被阻断时	ocn readiness list 看阻断项与 fix_hint
某条检查确实不适用本项目时	ocn readiness waive <checkId> --reason "..." --probe "..."
进入 build（11 之后）前最后过一遍	ocn readiness list

豁免示例：

```
ocn readiness waive rdy_network_engineer \
  --reason " 纯本地 CLI 工具，无网络架构" \
  --probe "test ! -d infra"
```

豁免的语义（开放世界、探针授予即验证并持续复验、状态切换即过期、人类专属且写入审计）以 3.13 为准，此处不重复。

2.5 第二阶段：Execution Navigator（11 之后）

不再以 advance 为主，改为按**任务循环 + 证据循环**。SOP 0.5.0 起，BUILD 态的核心节奏是逐个勾销任务台账：

```
ocn task list           # 看台账：哪些任务 pending、下一个是谁
/ocn-next              # 派出第一个未清任务（目标 = 任务规格原文）
# (AI 按 TDD 实现，范围限于任务 touches)
ocn task check          # 跑该任务冻结的验收命令；exit 0 才算 done
# (重复直到台账全清——否则 advance 不放行)
```

证据循环的分工不变。已接线 Claude Code 时：/ocn-next 一步替代 ①-④ 的”读现场 + 生成简报 + 派活”（next-prompt 内部本就读取 git/state/验收证据，①③ 降级为人想亲眼看证据时的查询工具）；⑤ 期间的编辑反馈与结束回合门禁由钩子自动执行；⑥⑦ 是人 review 用的收口工具，任何模式下都不被替代。一轮典型迭代：

```
ocn exec status         # ① 每轮开始：读本地 git 现场（分支/脏净/改动文件）
ocn github analyze-pr <n> # ② 有 PR 时：读 PR 元数据、checks、reviews
ocn evidence map [--pr <n>] # ③ 对照 03 验收标准：哪些 AC 有证据、哪些缺
ocn next-prompt         # ④ 生成下一轮 Agent (Claude Code / Codex) 任务简报
# (AI 执行一轮实现)    # ⑤
ocn verify status --mode combined --pr <n> # ⑥ 实现后：验证就绪度 ready/partial/blocked
ocn verdict draft       # ⑦ 收敛判断：继续开发/请求修改/可 review/可 merge
```

六条命令全部只读：不写 .ocoding/、不动 git/gh、不调 LLM。各命令的作用与使用时机：

- **ocn exec status** 读本地 git 证据（分支、head、脏/净、改动文件、近期 commit）与当前 OCN state——每轮迭代开始、或离开一段时间回来恢复现场时
- **ocn github analyze-pr <n>** 只读分析 PR 元数据、files changed、checks、reviews 并给出风险标记——PR 创建后、review 前、merge 决策前
- **ocn evidence map** 把 03 验收标准与本地/PR 证据逐条映射（evidence-found / candidate / missing / needs-human-review）——怀疑”做完了吗”的任何时刻
- **ocn next-prompt** 基于 state + git + 验收证据生成下一轮 Agent 任务简报（目标、允许的工作、禁止动作、验证命令、停止条件）——要派活给 AI 之前，代替手写 prompt
- **ocn verify status** 读 package scripts 与验证信号，汇总为 ready / partial / blocked / pending——每轮实现结束后；ready 才考虑 review
- **ocn verdict draft** 基于证据草拟阶段判断（继续开发 / 请求修改 / 可 review / 可 merge / 待人工）——里程碑收口时；结论给人裁定，不替人拍板（人机分工见 6.2/6.3）

2.6 回拨与重开：游标的受控移动

游标默认只进不退（advance 是唯一的前进方式），但现实中”回去”是常态，而且分两种性质完全不同的用途：

- **常规节奏**——里程碑衔接：完成边界含多个里程碑的项目，每个里程碑收口后回拨到 build plan 追加下一阶段任务（见下文里程碑循环）。这不是出错，是多阶段项目的标准走法。
- **异常恢复**——返工纠错：门禁通过后才发现上游文档要返工、中途 sop upgrade 越过了任务台账的生成点。

两种用途共用同一对受控命令，不必也**不准**手改 .ocoding/state.json（手改会绕过锁与审计，在审计链上留下解释不了的时间倒流）：

```
ocn rewind --to <stepId> --reason "< 为什么 >" # 轮内回拨：游标拨回严格更早的一步
ocn cycle new --yes # 跨轮重开：本轮收档归档，从头开新一轮
```

三条纪律，与 advance 完全对等：

- **回拨零豁免**——rewind 只移动游标，docs/ 一个字不动；之后每次 advance 都重过完整门禁（章节 + 逻辑 + 就绪 + 任务）。它把项目送回裁判面前，不替你过关。--reason 必填，全文进审计。
- **重开不丢历史**——cycle new 把本轮运行时状态归档进 .ocoding/cycles/< 轮次 >-< 时间戳 >/，docs/ 全部保留（下一轮门禁对着现成文档快进）；审计日志**不随轮归档**，一条链贯穿所有轮次。--yes 必填。
- **都是人类专属**——两条命令只在 CLI 存在，不暴露给 MCP/agent：“项目位于何处”的改写权不交给 AI。

典型场景对照：

场景	用哪条
升级 0.5.0 后发现游标已越过 build plan、台账没生成	ocn rewind --to step_build_plan --reason ... → 补 Task Specs → ocn check 生成台账
verify 阶段发现现实有缺口，要回 BUILD 返工 同一项目的下一个里程碑/阶段（完成边界未到）	ocn rewind --to <build 态某步 > --reason ... 里程碑循环 ：ocn rewind --to step_build_plan + 追加任务规格，见下文
项目完成边界达成（01 范围定义的最后一个里程碑收口）	ocn cycle new --yes 收档，开下一个项目周期

里程碑循环（实战验证的标准用法）：当 01 范围把完成边界定义为多个里程碑（如 P0→P4 各配一个 go/no-go 决策点）时，整个项目是**一轮**，每个里程碑收口走一次回拨循环：

```
# ① 人签本里程碑的 verdict (= 该阶段 go/no-go 裁定)，git commit 固化收据
# ② 回拨到 build plan
ocn rewind --to step_build_plan --reason " 里程碑 N 收口，追加 N+1 阶段任务规格"
# ③ 在 ## Task Specs 里【追加】下一阶段任务（phase 标签区分）
#   △ 已完成阶段的任务规格原文保留，verify 命令一字不动
# ④ 重过门禁——台账哈希对账
ocn check # verify 命令没变的任务保留 done，新任务进 pending
# ⑤ 正常 BUILD 循环：/ocn-next 派活 → ocn task check 勾销 → advance
```

这套循环的引擎保证（均有测试钉死）：台账重生成时 **id 与 verify 哈希双不变才保留 done**——所以第 ③ 步绝不要改动已完成任务的 verify 命令文本（改一个字符即重置 pending）；回拨后台账在 plan 态不会误触发派活（派发只认 state_build）；出 BUILD 守卫只认 pending。净效果：**一本台账累积整个项目的进度**（如 P0 的 19 done + P1 的 7 pending），每个 phase 的收据靠 git 历史留底。

cycle new 因此是**每个项目周期一次**的收档动作，不是每个阶段一次——轮内迭代是 rewind 的职权。

2.7 自动模式：把推进触发委托给 AI（可选，默认关闭）

前面所有流程的默认前提是：**推进由人按回车**——每过一道门禁，人来跑 ocn advance；每勾销一个任务，人来跑 ocn task check。门禁栈硬化到第四类假完成都堵上之后，瓶颈从”判定”变成了”触发”：判定已经是机器的，触发还是人的。

自动模式（对应 OCN 0.6.0-beta.0 / AM-009）让人**按阶段**把触发权交给 AI——但只交触发权，不交裁定权。一句话：

自动化的是按按钮的手，不是裁判。每次 advance 照样过完整门禁栈，任务完成照样只认冻结验收命令的 exit 0。

开关是人类专属的授权动作（拒绝 ai_agent 调用）：

```
ocn auto on --phase 1    # 委托第一阶段（DISCOVERY→PLAN 规划流水线）
ocn auto on --phase 2    # 委托第二阶段（BUILD→VERIFY，含 task check 与里程碑回拨）
ocn auto on --phase all  # 全自动：两阶段一起
ocn auto status          # 看当前授权 + 熔断状态
ocn auto off             # 回到全手动（默认）
```

阶段怎么分（按 advance 的目标态判定，零特判）：第一阶段 = 推进目标落在 DISCOVERY/SPEC/DESIGN/PLAN；第二阶段 = 落在 BUILD/VERIFY。所以 PLAN→BUILD 那一跨属第二阶段——只开第一阶段时，AI 走完规划会停在这道界等人。SHIP/REFLECT 永不委托。

开启后 AI 怎么跑（已接线 Claude Code 时，ocn agent setup 已把 OCN_ACTOR=ai_agent 注入 settings，下面的签名自动带上）：

```
OCN_ACTOR=ai_agent ocn advance --rationale " 背景:…; 依据: 门禁全绿; 操作:advance"
OCN_ACTOR=ai_agent ocn task check --rationale " 背景: 任务 X; 依据: 冻结命令 exit 0; 操作:check"
# 多里程碑项目: AI 自驱里程碑循环（唯一可委托的回拨）
OCN_ACTOR=ai_agent ocn rewind --to step_build_plan --reason "P0 完成, 追加 P1 任务"
```

四道护栏（任何模式都生效）：

- **判定权零让渡**——门禁栈、逻辑主干、就绪、冻结验收命令照常判定；自动模式只省掉人按回车。
- **熔断**——AI 在同一步骤连续 5 次（可配）门禁失败，自动模式自动暂停（审计记 actor=system），AI 被拒、人不受影响；ocn auto resume 解除。防的是无人值守时烧 token 空转。
- **决策痕迹**——AI 每次触发强制 --rationale（背景/依据/操作），引擎另记机器上下文（门禁结果、台账计数、冻结命令、耗时）；ocn auto trace 按时间线全程复盘。AI 敷衍理由也骗不过机器上下文。
- **硬禁区**——ocn auto 开关本身、readiness waive、cycle new、sop upgrade、override、非里程碑 rewind，任何模式都拒绝 AI；MCP 面零变化（还是 7 个工具，自动模式纯 CLI）。

怎么停：AI 的自动循环遇到任一条件就停下交还人——触发被拒（reason 以 automation_ 起头）、熔断暂停、推进目标越出被授权阶段、到了终点步且没有剩余里程碑。

渐进采用：默认手动 → 先开第一阶段（文档流水线化、代码仍人审）→ 信任建立后再开第二阶段或全自动。

2.8 全流程速查（一页版）

—— 装 ——

```
npm install -g o-coding-navigation && ocn --version
```

—— 起 ——

```
ocn init --tier minimal
# (编辑 .ocoding/config.yaml 登记 commands.build/test/lint/typecheck)
ocn readiness list          # 就绪基线
ocn agent setup             # 接线 Claude Code（钩子 + 契约 +/ocn-next）
git add .claude CLAUDE.md && git commit # 入库全队共享
```

—— 第一阶段（对 00→11 每份文档重复） ——

```
ocn brief → ocn doc create <type> → 写 → ocn check → ocn advance
# 已接线: Claude Code 里 /ocn-next 自动跑前两步, Stop 钩子强制 check
# exit 2 → 补章节/逻辑主干/任务规格; exit 1 → ocn readiness list → 补证据或 waive
# 11 build plan 必含 Task Specs 拆分; 门禁通过即冻结任务台账
```

—— 第二阶段（每轮迭代重复） ——

```
# 前半·读现场 + 派活（二选一）:
# 手动:   ocn exec status → ocn evidence map → ocn next-prompt → 贴给 AI
# 已接线: /ocn-next 一步替代 (next-prompt 内部已读 git/state/验收证据)
# AI 干活 ——已接线时编辑反馈 + 结束回合门禁由钩子自动执行
# BUILD 态任务循环（0.5.0 核心节奏）:
```

```

ocn task list → /ocn-next 派任务 → AI 实现 → ocn task check 勾销
# 台账不清, advance 不准出 BUILD
# 收口·人裁定 (两种模式相同, 不被 /ocn-next 替代):
ocn verify status → ocn verdict draft → 人 review → ocn advance

# —— 回拨与重开 (人类专属, 不暴露给 agent; 里程碑衔接 + 返工均用此对命令) ——
ocn rewind --to <step> --reason "..." # 轮内回拨; 零豁免, advance 重过全部门禁
ocn cycle new --yes # 终点后开新一轮; docs 保留, 审计单链贯穿

# —— 自动模式 (可选, 默认关闭; 开关人类专属, 详见 2.7) ——
ocn auto on --phase 1|2|all # 按阶段把推进触发委托给 AI
ocn auto status / trace # 看授权 + 熔断 / 复盘每条 AI 决策
ocn auto resume / off # 熔断解除 / 回到全手动
# 开启后 AI 以 OCN_ACTOR=ai_agent + --rationale 触发 advance / task check / 里程碑 rewind
# 判定权不让渡; 熔断与硬禁区始终生效

# —— 升级 ——
npm install -g o-coding-navigation@latest && ocn sop upgrade

# —— 卸 ——
npm uninstall -g o-coding-navigation

```

最小工作闭环: 用 OCN 完成 00→11 并冻结任务台账 → Claude Code 按任务规格实现、git 与 GitHub 形成证据链 → OCN 读证据、生成下一轮 Agent 提示、归纳 verification 与 verdict → 人裁定是否 review、merge、发布。

第三章 两阶段总览 (Planning Gate · Execution Navigator)

3.1 总体结构

本版 SOP 分为两个连续阶段。

第一阶段: Planning Gate

从 00 到 11。

特点:

- 线性推进
- 强门禁
- 强结构化
- 以文档和定义为主
- 目标是形成可执行的 Build Plan

第二阶段: Execution Navigator

从 11 之后进入真实开发。

特点:

- 非线性
- 以证据链为主
- 以状态判断和下一轮行动建议为主
- 不再以手工推进线性文档为核心
- 目标是形成证据充分的实现、验证与最终判断

3.2 两阶段之间的衔接点

11-build-plan.md 是衔接点。

它意味着：

- 前半段的定义已经足够支撑开工
- 后半段要开始进入真实实现循环
- 开发不再只是“继续写文档”
- 开发开始变成“读证据、控偏差、促收敛”

3.3 角色变化

在第一阶段

OCN 是 **Planning Gatekeeper**

它负责：

- 要不要进入下一步
- 当前文档是否具备最低完整性
- 哪些定义还缺失
- 是否具备开工条件

在第二阶段

OCN 是 **Execution Evidence Navigator**

它负责：

- 当前实现做到哪里
- 哪些证据已经具备
- 哪些验收条目仍缺证据
- 当前是继续开发、请求修改、进入 review，还是等待人工判断
- 下一轮 Claude Code、Codex、LFG 应该做什么

3.4 第一阶段：Planning Gate

第一阶段的目标，是把项目从模糊语言，推进到可执行工程输入。

3.5 00 项目简报

文件：

docs/00-project-brief.md

目标：

- 定义问题
- 定义目标
- 定义用户
- 定义成功标准

如果这一层不清楚，后续所有内容都会漂移。

3.6 01 范围

文件：

docs/01-scope.md

目标：

- 明确 in scope
- 明确 out of scope
- 明确技术约束
- 明确本轮完成边界

这一层的价值，是控制贪婪扩张。

3.7 02 PRD

文件：

docs/02-prd.md

目标：

- 把产品需求转成结构化功能描述
- 明确对象、行为、输入、输出、边界
- 让后续验收与实现有对照基线

3.8 03 验收标准

文件：

docs/03-acceptance-criteria.md

目标：

- 把“做出来”变成“可判断是否做对”
- 把模糊目标转成验收项
- 为后续 evidence map 提供基线

3.9 04 技术架构

文件：

docs/04-technical-architecture.md

目标：

- 确定最终技术选择
- 明确运行形态、语言、存储、部署、约束与风险
- 形成架构决策锚点

3.10 05 到 09 设计层文档

- docs/05-information-architecture.md
- docs/06-data-model.md
- docs/07-logic-backbone.md（逻辑主干·见 3.11）
- docs/08-api-contract.md
- docs/09-test-strategy.md

目标分别是：

- 信息结构
- 数据结构
- 逻辑主干（计算/决策图）
- 接口契约
- 测试策略

3.11 逻辑主干 (Logic Backbone)

文件：

docs/07-logic-backbone.md

这是 v3 在设计层新增的强门禁产物。前面的设计层文档解决”系统有哪些东西、长什么样”，逻辑主干解决”这些东西按什么顺序、以什么角色连起来”。

什么时候写：设计层中段——在 06 数据模型写完之后、08 接口契约与 09 测试策略之前。依赖关系决定了这个位置：逻辑主干的输入/分数是**从数据模型的字段算出来的**（所以排在数据模型之后）；而接口契约暴露的是它算出来的结果、测试策略测的是它这张图（所以排在它们之前）。整个第一阶段（Planning Gate）必须在进入 build 之前把它接好、六类缺陷清零。

关于文件号 07：在 OCN 0.3.0 里它就是 docs/07-logic-backbone.md——**文件号等于书写顺序**（第 7 份产出），从 08 接口契约起的文档相应顺移。号即顺序，不再有”号在后、却要早写”的歧义。

做法

把系统的计算/决策语义建成一张**有类型、有角色的有向图（DAG + DMN 决策分层）**：

- **节点 kind：**input（输入/指标）/ formula（公式）/ score（分数）/ judgment（判断）/ signal（信号）
- **节点 role（必填）：**input / intermediate（中间，要被消费）/ terminal_explanatory（终点，只解释）/ trigger（触发，驱动功能）/ hint（提示，只告知）
- **边（上游 → 下游）：**feeds（算序）/ serves（公式 → 判断）/ feeds（分数 → 下层）/ triggers（信号 → 功能）/ explains（只解释、只提示）

它必须显式回答四件事：执行顺序、公式归属、分数分层、信号作用级别。

怎样算”接好线”——可机器校验

逻辑主干不是写一段叙事就算完，它是单一事实源、可被机器校验。命中以下任一硬缺陷即不通过：

1. 缺角色
2. 重复节点 id
3. 悬空引用（边指向未定义节点）
4. 依赖环（feeds/serves/triggers 子图成环）
5. 孤儿节点（input/intermediate 无下游消费）
6. 未绑定触发（trigger 没有 triggers 边指向已定义目标）

在 OCN 0.3.0 中，这一原则已产品化为 artifact_logic_backbone: ocn check 对以上六类缺陷返回 blocked（退出码 2）并逐条点名；通过后把规范化的图写入 .ocoding/logic-graph.json，由 ocn brief 注入”执行顺序 + 触发清单”摘要，使第二阶段不再漂移。范式参考 dbt 的 ref-DAG、DMN 决策需求图、Event Modeling、架构适应度函数。

3.12 11 Build Plan

文件：

docs/11-build-plan.md

这是第一阶段的终点，也是第二阶段的入口。

它的意义不是“又多一份文档”，而是：

- 把前半段的定义压缩成可执行计划
- 为执行阶段提供任务切分依据
- 为后续证据分析提供对照表

任务主干 (Task Backbone, v7 新增) ——build plan 的机器可解析部分

自 SOP 0.5.0 起, build plan 必须包含 ## Task Specs | 任务规格 章节: 把实施拆分成任务规格块, 每个任务是一份可证伪的迷你 spec:

```
### task_phase0_runtime_skeleton
- goal: permit runtime 最小可运行骨架 (替换全部 NotImplementedError)
- traces: AC-03, AC-07          ← 必须解析到 03 验收标准里的真实 AC
- touches: score_risk          ← 引用逻辑主干节点 (悬空即阻断)
- verify: pytest tests/test_runtime.py -q ← 本任务自己的确定性验收命令
- dod: 接口全部可执行; RED→GREEN 过程记入 12 号
```

(可选字段: depends (任务依赖, 禁止成环)、phase (分组)、timeout (验收命令秒数上限))

六类硬缺陷, 任一命中即 build plan 门禁阻断 (exit 2): 重复/非法任务 id、必填字段缺失、traces 悬空、touches 悬空、depends 悬空或成环、零任务 (有章节没拆分)。

门禁通过的瞬间: 任务台账 .ocoding/task-ledger.json 生成, 每个任务的验收命令**哈希冻结**进台账 (裁判不在选手写路径上) ——实施期间改 build plan 必须重过门禁。

拆分准则: 一个任务 = 一次 PR 级增量 (diff ≤500 行)、有自己独立可跑的验收命令。拆分质量由人 + AI 在写 build plan 时把控, 门禁负责证伪。

3.13 就绪主干 (Readiness Backbone)

这是 v5 新增的**横切门禁**——它不是一份新文档、不是一个新 step, 而是一套随 SOP 打包的**就绪规则手册**, 在每次 ocn check / ocn gate / ocn advance 时, 于章节门禁与逻辑主干门禁之后自动运行, 贯穿两个阶段。

它防的是什么

第三类假完成: **角色盲区的假完成**。文档结构齐全 (第一类已被章节门禁堵住)、逻辑也接线了 (第二类已被逻辑主干堵住), 但项目对”必须验收它的那批人”来说根本没准备好——没有版本控制、没有可复跑的测试命令、没人对成本和可运维性负责、没有真实使用者。

54 个角色从哪来——角色编目

54 个角色不是拍脑袋列的, 而是取自外部成熟编目 (基于 APQC PCF / ITIL 的 IT 角色知识库), 按职能分**四层**, 每个角色至少对应一条验收关注点, 共 **55 条可证伪的就绪检查**:

层	角色数	典型角色	典型检查关注点
战略层 (strategy)	10	CIO、IT 战略规划、业务关系经理、企业架构师	有没有真实使用者、价值假设、成本测算、是否”准备过头”
架构层 (architecture)	8	解决方案架构师、数据架构师、安全架构师	架构决策有锚点、数据模型与接口契约一致、安全基线
交付层 (delivery)	15	开发、QA、DevOps、业务分析、项目经理	有版本控制与 CI、测试可复跑、验收标准可证伪、变更有证据
运营层 (operations)	21	SRE、服务台、安全运营、容量/可用性管理	有人对可运维性负责、监控与告警、故障与回滚路径

分级 (tier) ——不同规模要求不同的角色子集

不是每个项目都要伺候全部 54 个角色。每条检查声明它在哪些 tier 下是必需的 (由该角色通常出现的最小团队规模推导), 项目在 ocn init --tier <t> 时定级:

init --tier	就绪 tier	含义	必需检查范围
minimal	solo	单人/极小项目	最小集：开发、QA、DevOps、基本安全与使用者等核心角色
production	team	小团队、要上生产	solo 全集 + 项目管理、架构、成本、合规等团队级角色
full	platform	平台级/多团队	全部 54 角色，含治理、容量、连续性等平台级角色

不属于当前 tier 的检查自动判 **NA**（不算缺漏，不阻断）；属于当前 tier 的检查才参与门禁。tier 在 init 时随探针命令一起 hash 冻结（R4），中途偷偷调低定级以绕过检查会被发现。

每条检查怎么取证与判定

- **确定性取证**：通过文档别名（doc slug 通配）与仓库探针（文件存在性、config.yaml 里登记的 build/test 命令）解析证据，不调用 LLM，本地优先
- **开放世界语义**：block 级且当前 tier 必需的检查，只有 PASS 或 WAIVED 算过——**FAIL 和 UNKNOWN 一样阻断**（沉默不是通过）；阻断时返回 ERR_GATE_FAILED（退出码 1）并逐条给出双语 fix_hint
- **warn 级检查**只进 ocn brief 提示，不阻断（如战略层的”准备过头”检查）
- **判定台账**：每轮评估写入 .ocoding/readiness.json，ocn brief 把未决项 + fix_hint 列成 BUILD 工作清单

豁免的语义

确实不适用的检查走**有条件豁免**（waive-with-probe）：授予时探针必须通过（不发”死豁免”），之后每次门禁运行都复验探针，项目离开当前状态时豁免自动过期。豁免是人类专属操作（不暴露给 MCP），每次授予都写入审计。

具体命令与使用时机统一见第二章（2.4）；旧项目迁移见 2.1。

在 OCN 中，这一原则自 0.4.0 起已产品化为 readiness 横切门禁。设计灵感来自 readiness review / Definition of Done / 生产就绪评审（PRR）传统。

3.14 为什么第二阶段不能继续做成线性文档推进

真实开发后半段最常见的节奏不是：

- 写完 12
- 再写 13
- 再写 14
- 再写 15

真实节奏往往是：

- 改一轮代码
- 跑一轮测试
- 遇到问题
- 修一轮
- 补一轮证据
- 看 PR 状态
- 再生成下一轮提示词
- 再验证
- 再判断是否收敛

所以，第二阶段不能继续以手工 advance 若干文档为主。

3.15 第二阶段的事实来源

后半段最重要的事实不在文档里，而在工程证据里：

- 本地 git 状态
- 当前分支
- commit 历史
- changed files
- GitHub PR 元数据
- checks 状态
- review 结论
- acceptance evidence coverage
- verification summary

第二阶段的六条核心命令（exec status / analyze-pr / evidence map / next-prompt / verify status / verdict draft）的清单、作用与使用时机，统一见第二章（2.5）。

3.16 第二阶段不是取消文档，而是改变文档角色

11 到 18 仍然可以存在，但其角色发生了变化。

它们不再主要是“用户手工推进的表单”，而更适合变成：

- 对执行证据的汇总视图
- 对 GitHub、git、CI 状态的结构化归纳
- 对最终 verdict 的人类可读报告

也就是说：

后半段是证据驱动，文档承接，而不是文档驱动，证据补写。

第四章 完整文档体系

4.1 第一阶段核心文档

- 00 project brief
- 01 scope
- 02 PRD
- 03 acceptance criteria
- 04 technical architecture
- 05 information architecture
- 06 data model
- 07 logic backbone（逻辑主干 · 计算/决策图，可机器校验 · 数据模型之后、接口/测试之前）
- 08 API contract
- 09 test strategy
- 10 MVP plan
- 11 build plan

4.2 第二阶段可承接文档

- 12 implementation log
- 13 change evidence
- 14 integration notes
- 15 verification report
- 16 acceptance mapping
- 17 failure fix log

- 18 regression evidence
- 19 final build verdict

4.3 本版推荐理解

00 到 11 是 **强门禁输入体系**。
12 到 19 是 **执行证据承接体系**。

第五章 阶段门禁与推进规则

5.1 第一阶段推进规则

第一阶段以门禁线性推进（check → gate → advance 循环，命令见 2.3），因为这部分内容具有天然顺序。

没有定义，就不该设计。

没有验收，就不该实现。

没有架构，就不该拆数据和接口。

没有测试策略，就不该进入 build。

自 v5 起，每道门禁在章节检查与逻辑主干检查之后，还会运行**就绪检查**（3.13）：当前 tier 必需的 block 级检查必须 PASS 或被显式豁免，否则 advance 不放行。补证据（按 fix_hint）或显式豁免（2.4）是仅有的两条出路——没有第三条”装作没看见”。

5.2 第二阶段推进规则

第二阶段不要再把 advance 当作主要交互方式，改为证据循环（命令见 2.5）：

- 先读现场
- 再看 PR
- 再看验收证据
- 再生成下一轮提示
- 再看验证状态
- 再形成 verdict

5.3 判断逻辑

继续开发

当证据仍不足、验证未完成、实现未收敛时。

请求修改

当 evidence 已足够指出明显缺口、失败或不一致时。

进入 review

当主要验证通过、证据已基本具备，但还需要人工审查时。

准备 merge

当 PR clean、checks success、evidence 足够、无明显阻断项时。

等待人工判断

当 qualitative criteria、多义风险或冲突证据存在时。

第六章 AI 与人的分工

6.1 AI 适合做什么

- 根据结构生成初稿
- 解析 git 与 GitHub 证据
- 归纳当前状态
- 识别缺失证据
- 生成下一轮 prompt
- 草拟 verification summary
- 草拟 verdict draft

6.2 人必须保留什么

- 范围裁定
- 关键取舍
- 风险接受
- 最终验收
- 是否发布
- 是否 merge
- 是否进入下一阶段
- 是否开启自动模式、对哪个阶段授权、熔断后是否恢复 (ocn auto, 开关本身人类专属)

注：默认手动模式下，状态推进 (ocn advance) 由人保留。开启自动模式 (2.7) 后，被授权阶段内的推进**触发**委托给 AI，但**判定权不让渡**——门禁与冻结验收命令照常裁定，熔断与硬禁区兜底。ocn cycle new、ocn sop upgrade、ocn readiness waive 与非里程碑 ocn rewind 在任何模式下仍为人类专属。

6.3 不应让 AI 做什么

- 自主发布 npm
 - 自主移动 latest
 - 自主打 tag
 - 自主创建 release
 - 自主宣称 GA
 - 自主扩大修改范围
 - 在证据不足时做确定性判断
-

第七章 常见错误与纠偏建议

7.1 错误一：还没定义清楚就急着进入实现

纠偏：

先回到 00 到 11，补齐 Planning Gate。

7.2 错误二：进入开发后继续把 OCN 当作文档推进器

纠偏：

转入 Execution Navigator 交互模型。

7.3 错误三：重复手工记录 GitHub 已经存在的证据

纠偏：

优先读取工程事实，减少重复抄写。

7.4 错误四：把 AI 当作最终裁判

纠偏：

AI 负责归纳和建议，人负责裁定和授权。

7.5 错误五：把 evidence-candidate 当成 evidence-found

纠偏：

保持保守。

证据不足时，不要假装已经完成。

7.6 错误六：在 partial 状态下强行进入 merge

纠偏：

先补 evidence，再补 verification，再判断是否 ready。

7.7 错误七：文档结构齐全，但逻辑未接线

模块、指标、公式都写齐了、章节也都在，但运行时的计算-决策语义从没被显式接线——这是”结构齐全但逻辑未接线”的假完成。

纠偏：

在设计层先产出逻辑主干（3.11），把执行顺序、公式归属、分数分层、信号作用级别显式画成可机器校验的图；六类缺陷（缺角色/重复/悬空/环/孤儿/未绑定触发）不清零，不进入 build。

7.8 错误八：角色盲区的假完成

每道门禁都过了、逻辑也接线了，可项目对真正要验收它的人来说没准备好——没有 git/CI、没有可复跑的测试命令、没有使用者、没人对成本和可运维性负责。多角色评审一问就穿。

纠偏：

让就绪主干（3.13）持续运行：ocn readiness list 看当前 tier 还有哪些角色检查是 FAIL/UNKNOWN，按 fix_hint 补证据；确实不适用的，用 ocn readiness waive 显式豁免并留下理由与探针。**沉默不是通过，豁免必须留痕。**

7.9 错误九：收据型假完成与全链空转

BUILD 的三份收据文档如实写”本阶段无代码变更”，章节齐全、门禁全过；继续 advance，整条 SOP 走完一轮而实现从未被排入——状态机成了跑步机（v7 修订的触发现场）。

纠偏：

升级到 SOP 0.5.0, 让任务主干 (3.12) 接管: build plan 必须拆出任务规格, 勾销只认每个任务冻结的验收命令, **任务台账不清不准出 BUILD**。如果已经空转到 verify 中段: 用 `ocn rewind --to step_build_plan --reason "..."` 把游标受控拨回 build plan (2.6), 补 Task Specs、重过门禁生成任务台账, 再按任务循环逐个勾销——回拨进审计、零豁免, 比”游标驻留原地手工补”更可追溯。状态机是门卫, 不是跑步机, 没有任何规则要求你持续 advance。

第八章 修订说明

本版是在原有 AI Coding 开发 SOP 基础上的一次关键升级。

原始 SOP 的优势非常明显:

- 它把开发前的混乱问题, 转化成了可结构化的定义问题
- 它把需求、边界、架构、接口、测试策略前置
- 它让 AI Coding 不再从一团模糊语言中开始
- 它让项目在开工前就形成了最低限度的工程秩序

但在真实 dogfood 过程中, 也暴露出一个重要问题:

前半段顺畅, 进入开发后容易卡住。

原因并不复杂。

开发前半段适合线性推进。

真实开发后半段并不适合线性推进。

在开发阶段, 真正发生的事情是一个非线性循环: 改代码 → 跑测试 → 遇错 → 修复 → 补证据 → 再验证 → 处理 PR 与 review → 对照验收判断是否收敛 (详见 3.14)。

所以, 后半段最自然的事实来源, 不是再手工维护一套线性文档, 而是已经存在的工程证据链——本地 git、GitHub PR、commit、diff、review、CI、test result、acceptance evidence、verification summary (详见 3.15)。

因此, 本版 SOP 引入 OCN 的两阶段模型:

阶段一

Planning Gate

用于锁定输入、边界、架构、验收与计划。

阶段二

Execution Navigator

用于读取执行证据、判断状态、归纳偏差、生成下一轮 Agent 指南。

v3 升级: 补上”逻辑主干”

v2 之后在 dogfood 中又暴露出另一个隐蔽问题:

第一阶段的文档结构很齐全, 但系统的”计算/决策逻辑”在设计时从没被文档明确。

模块名、指标、公式都列全了, 可真正运行时的计算-决策语义 (先算什么、哪个公式服务哪个判断、哪个分数进下一层、哪个信号触发功能) 从没被显式接线——它散落在多份文档的散文里、藏在各人脑子里, 于是进入开发后**不断漂移, 越往后越乱**。

这是一类新的”假完成”: **结构齐全, 但逻辑未接线。**

因此 v3 在第一阶段的设计层补上一份新的强门禁产物——**逻辑主干 (Logic Backbone)**：把系统的计算/决策语义在开工前建成一张有类型、有角色、可机器校验的有向图（详见 3.11）。它对应 OCN 0.3.0 的 `artifactual_logic_backbone`。

v4 修订

v4 不改方法论，只做可读性收口：把重复表述各归并到唯一规范主场——逻辑主干四要点统一由 3.11 规范、后半段开发节奏统一由 3.14 叙述、第二阶段事实来源统一由 3.15 列举；本章只保留概述并指向它们。同时新增第零章安装指南。

本次 v5 修订：补上”就绪主干”

逻辑主干堵住了”结构齐全但逻辑未接线”之后，dogfood 中又暴露出第三类更隐蔽的假完成：

几十份设计文档每道门禁全过，可一场多角色评审仍能当场问出基本缺口——没有 git/CI、没有使用者、没有成本测算、没人对可运维性负责。

原因是：原有验证面是**文档内的**（一份文档对一套 SOP 模式）、**封闭世界的**（没写 ≠ 不通过）。它检查”该写的章节写了没有”，却从不检查”这个项目对必须验收它的那批人来说，准备好了没有”。

“缺了哪些维度”无法穷举；但”哪些角色必须签收”是有边界、可外部编目的。所以 v5 把不可验证的命题（内容完整吗？）转换成可验证的命题（**每个必需验收角色都 PASS 或被显式豁免了吗？**）——这就是**就绪主干 (Readiness Backbone)**：基于 54 个 IT 角色编目的 55 条可证伪就绪检查，作为横切门禁贯穿两个阶段（详见 3.13）。它对应 OCN 0.4.0 的 readiness 横切门禁，并配套 `ocn sop upgrade` 让旧项目一键迁移。

同时，v5 新增**完整操作指引**一章（现第二章）——全书唯一的命令主场：安装/升级/卸载、项目启动、第一阶段、就绪主干、第二阶段的逐条命令与使用时机，外加一页速查。v4 的第零章安装指南随之并入 2.1，其余各章不再罗列命令、只讲原理并指向该章。

本次 v6 修订：把接线产品化——纪律靠机械强制，不靠记忆

v5 之前，本 SOP 与 Claude Code 之间的接线是一份”手工 runbook”：自己配 hooks、自己维护治理契约、每个任务手跑 `brief / next-prompt` 再粘贴提示词。手工接线必然漂移、必然被跳过——**OCN 无法机械强制的纪律，等于不存在的纪律**（这正是本产品的立论）。

v6 把 runbook 产品化进 OCN（对应 0.4.0-beta.2 的 `ocn agent setup` 与 `ocn hook` 命令）：

- **一条命令完成接线**：`ocn agent setup` 生成 hooks、治理契约、`/ocn-next` 斜杠命令（详见 2.2）；文件入库后全队克隆即生效
- **结束回合即门禁**：agent 想结束回合时自动重跑 `ocn check`，没过就把 `fix hints` 顶回去继续修——绕不过去
- **改完即反馈**：每次编辑文件后自动跑 `lint/typecheck`，错误直接回灌给 agent 当场修
- **每任务两个人类动作**：Claude Code 里 `/ocn-next`，终端里 `review + ocn advance`——中间的纪律全部机械接管

本次 v7 修订：补上”任务主干”——堵死第四类假完成

v6 接线后第一轮真实 dogfood，当天就暴露了第四类、也是最隐蔽的假完成：**收据型假完成**。BUILD 阶段的三份文档（实现日志/变更证据/集成说明）是实施的”收据”，引擎只验收收据的章节、不验收收据背后的现实——于是 agent 如实交出”零代码收据”，每道门禁全过；更严重的是**全链空转**：整条 SOP 几乎诚实地走完一轮，而一行业务代码从未被任何步骤排入。状态机成了跑步机。

v7 的解法与前两根主干同构——把不可验证的命题（“按计划实现了吗？”）换成可验证的命题（“**每个任务规格的验收命令都跑通了吗？**”）：**任务主干 (Task Backbone)**。build plan 内嵌机器可解析的任务规格块（每个任务 = 一份迷你 spec），门禁校验六类硬缺陷并把验收命令哈希冻结进任务台账；勾销只认验收命令、台账不清不准出 BUILD（详见 3.12）。它对应 OCN 0.5.0 的 task 门禁与 `ocn task` 命令。

至此三根主干集齐：**逻辑主干管”接没接线”，就绪主干管”角色签没签收”，任务主干管”实现真没真发生”——四类假完成全部有机械防线。**

本次 v8 修订：补上”回拨与重开”——受控逃生通道

三根主干集齐后，dogfood 当天暴露了最后一块缺口：游标只进不退。中途升级越过任务台账生成点的项目无法回去重过门禁；走到终点步的项目没有体面的重开方式；于是**手改 .ocoding/state.json 成了事实上的唯一逃生通道**——绕过锁与审计，恰恰违反本 SOP 自己卖的纪律。没有受控逃生通道，使用者就会用不受控的方式逃生。

v8 补上两条人类专属命令（对应 OCN 0.5.0-beta.1 的 DEC-033）：ocn rewind 轮内回拨（强制说明理由、零豁免、全程审计——时间线永远向前，游标可以向后）与 ocn cycle new 跨轮重开（本轮归档、docs 保留快进、审计单链贯穿所有轮次）。终点步的报错信息同步指路这两条出路。详见 2.6；7.9 的空转纠偏路径随之更新为机械可追溯的回拨方案。

本次 v9 修订：里程碑循环——回拨的标准用法定型

v8 落地当天的 dogfood 把回拨用出了比预想更准的形态：对”完成边界 = 多个里程碑”的项目（P0→P4 各配 go/no-go），**每个里程碑收口用 rewind 拨回 build plan、追加下一阶段任务规格、重过门禁**——台账哈希对账自动保留已勾销任务，一本账累积整个项目进度（实战现场：19 done + 7 pending 共 26 任务）。cycle new 的定位随之收窄为**项目周期级**收档（完成边界达成后用一次），轮内迭代全部归 rewind。v9 把这套里程碑循环写进 2.6 并给出引擎保证清单；不新增任何命令或功能。

本次 v10 修订：术语校正——“纠错”改”回拨与重开”

v9 把回拨章节命名为”纠错”，实践立即证明这个标签太窄：P0→P1 的里程碑衔接用的就是 rewind，但它不是纠错，是多阶段项目的常规节奏。v10 将 2.6 标题与速查段标签改为中性的**“回拨与重开”**，并在章节开头明确两种用途（常规节奏的里程碑衔接 / 异常恢复的返工纠错）。无功能变化。

本次 v11 修订：自动模式——可选的触发权委托

三主干 + 回拨/重开集齐后，门禁栈硬化到四类假完成全部有机械防线；dogfood 随之暴露新瓶颈：判定已经是机器的，触发还是人的——每过一道门禁、每勾销一个任务都要人按回车。

v11 引入**可选自动模式**（对应 OCN 0.6.0-beta.0 / AM-009 / DEC-034）：人通过人类专属的 ocn auto 开关按阶段授权后，AI 可在被授权阶段内自主触发 advance、task check 与里程碑回拨。核心立场是把”advance 是人类专属”改述为”advance 是人类**授权**”——委托的是触发权，判定权（门禁栈、冻结验收命令的 exit 0）一寸未让。四道护栏保住纪律叙事：默认手动、开关人类专属、熔断（连续失败自动暂停、人不受影响）、硬禁区（waive/cycle/sop upgrade/override/非里程碑 rewind/开关本身永不委托）；外加强制 --rationale + 引擎机器上下文 + ocn auto trace 让每条 AI 决策可复盘。多 P 构建计划由 AI 自驱里程碑循环一跑到底。引擎/CLI 特性，不 bump SOP 版本；MCP 白名单 7 工具不变。详见 2.7。